

# Byzantine-Tolerant Decentralized Proximal Gradient Methods

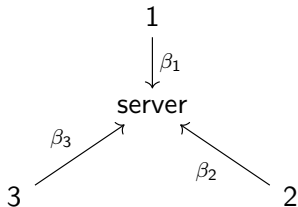
Gilbert Paul Neuner

May 19, 2026

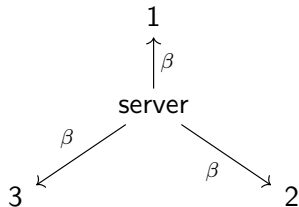
- 1 Background
- 2 Main Algorithm
- 3 Convergence Analysis
- 4 Numerical Experiments
- 5 Future Research Plans

# What is federated learning?

- Federated learning (FL) is a machine learning technique in which multiple clients collaborate to train a model while keeping their local datasets private.
- FL is exemplified by FedAvg [MMR16], which Google uses for next-word prediction on Android Gboard.



(a) Clients broadcast  
 $\beta_i := \beta - \nabla f_i(\beta)$  to server



(b) Server broadcasts  
 $\beta := \frac{\beta_1 + \beta_2 + \beta_3}{3}$  to clients

Figure: A round of FedAvg

Our main contribution is a *Byzantine-tolerant DFL* method characterized additionally by the following features:

- We give five versions of the main algorithm, each corresponding to a “model update sequence”.
  - To the best of our knowledge, two of these sequences are new, and one has only been used in CFL methods.
- Prior to aggregation, the magnitudes of the input vectors are “clipped”.
- During aggregation, clients weight the contributions of the neighbors based on “trust scores”.

## 1 Background

- Federated Learning
- Centralized vs. Decentralized FL
- Byzantine-Tolerance

# Setup

- In **federated learning (FL)**, there are  $K$  **clients**, labeled by the members of the index set  $[K] := \{1, \dots, K\}$ .
- Each client  $i \in [K]$  possesses a **local dataset**  $D_i = \{(x_{il}, y_{il})\}_{\ell=1}^{n_i}$ .
- Local datasets are **private**, i.e., clients will not exchange local datasets.
- The  $i$ -th **local objective function** is

$$f_i(\beta) = \frac{1}{n_i} \sum_{\ell=1}^{n_i} \phi(\beta; x_{il}, y_{il}).$$

- The parameter vector  $\beta \in \mathbb{R}^p$  is called a **model**.
  - The function  $\phi$  is a data point specific loss function.
- The goal of FL is to minimize the **global objective function**

$$f(\beta) = \frac{1}{K} \sum_{i=1}^K f_i(\beta).$$

# A round of FL

- During round  $m$ , client  $i$  computes  $g_i^m$ , its **local stochastic gradient**, which estimates its **local gradient**  $\nabla f_i(\beta_i^m)$ , using a mini-batch from  $D_i$ .
- Having **fetched** messages  $g_j^m$  and/or  $\beta_j^m$  from neighbors  $j \in \mathcal{N}_i$ , client  $i$  **aggregates** messages  $g_j^m$  and/or  $\beta_j^m$ , i.e., takes a weighted sum, e.g.

$$\sum_{j \in \mathcal{N}_i} w_j^m g_j^m \text{ or } \sum_{j \in \mathcal{N}_i} w_j \beta_j^m.$$

- Having done aggregation, client  $i$  performs **model update(s)**, i.e., assigns to  $\beta_i^{m+1}$  an updated model based on the current model  $\beta_i^m$  and the aggregated message, e.g.

$$\beta_i^{m+1} \leftarrow \beta_i^m - g_i^m$$

$$\text{or } \beta_i^{m+1} \leftarrow \beta_i^m - \sum_{j \in \mathcal{N}_i} w_j^m g_j^m$$

$$\text{or } \beta_i^{m+1} \leftarrow \sum_{j \in \mathcal{N}_i} w_j^m \beta_j^m.$$

## 1 Background

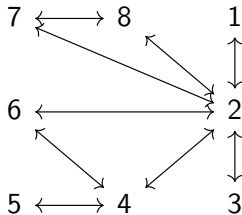
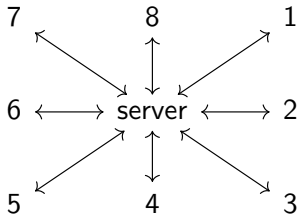
- Federated Learning
- Centralized vs. Decentralized FL
- Byzantine-Tolerance



# CFL vs. DFL

We note the following differences between **CFL** and **DFL**:

- In CFL all clients learn the same **global model**  $\hat{\beta}$ . In DFL each client will learn a distinct **local model**  $\hat{\beta}_i$ .
- In CFL there is a **server** that does not possess a local dataset but performs aggregation and model updates. In DFL there is no server and clients perform aggregation and model updates themselves.
- CFL occurs on the **star-shaped graph**, with the server in the middle. DFL occurs on an arbitrary graph.



The two most basic examples of CFL are FedAvg and FedSGD [MMR16]. Training loop of **FedAvg**:

- ①  $\mathbf{g}_i^m \leftarrow \nabla f_i(\beta^m)$  (clients estimate local gradients)
- ②  $\beta_i^{m+1/2} \leftarrow \beta^m - \eta^m \mathbf{g}_i^m$  (clients do GD-like update)
- ③  $\beta^{m+1} \leftarrow \frac{1}{n} \sum_{i=1}^K n_i \beta_i^{m+1/2}$ . (server does FedAvg-like update)

Training loop of **FedSGD**:

- ①  $\mathbf{g}_i^m \leftarrow \nabla f_i(\beta^m)$  (clients estimate local gradients)
- ②  $\beta^{m+1} \leftarrow \beta^m - \frac{\eta^m}{n} \sum_{i=1}^K n_i \mathbf{g}_i^m$ . (server does FedSGD-like update)

The two most basic examples of DFL are D-SGD and D-PSGD.

Training loop of **D-SGD** [KLB21] ( $W$  is a weight matrix):

- ①  $g_i^m \leftarrow \nabla f_i(\beta_i^m)$  (clients estimate local gradients)
- ②  $\beta_i^{m+1/2} \leftarrow \beta_i^m - \eta^m g_i^m$  (clients do GD-like update)
- ③  $\beta_i^{m+1} \leftarrow \sum_{j \in \mathcal{N}_i} W_{ij} \beta_j^{m+1/2}$ . (clients do FedAvg-like update)

Training loop of **Decentralized Parallel SGD (D-PSGD)** [LZZ17]:

- ①  $g_i^m \leftarrow \nabla f_i(\beta_i^m)$  (clients estimate local gradients)
- ②  $\beta_i^{m+1/2} \leftarrow \sum_{j \in \mathcal{N}_i} W_{ij} \beta_j^m$  (clients do FedAvg-like update)
- ③  $\beta_i^{m+1} \leftarrow \beta_i^{m+1/2} - \eta^m g_i^m$ . (clients do GD-like update)

- 1 Background
  - Federated Learning
  - Centralized vs. Decentralized FL
  - Byzantine-Tolerance

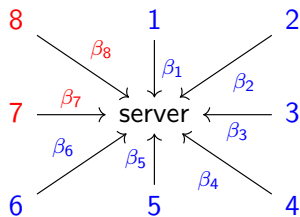
# Byzantine failures

- In the Byzantine setting, clients are either **honest** or **Byzantine**.
- The set of honest neighbors of client  $i$  is denoted by  $\mathcal{H}_i$  and the set of Byzantine neighbors of client  $i$  is denoted by  $\mathcal{B}_i$ .
- In the Byzantine setting, client  $i$  fetches from neighbor  $j$

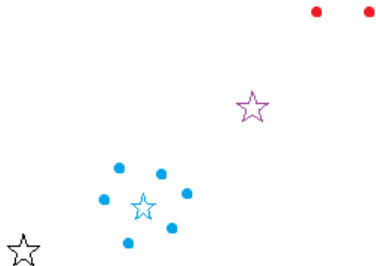
$$g_{ij}^m = \begin{cases} g_j^m, & j \in \mathcal{H}_i \\ *, & j \in \mathcal{B}_i, \end{cases}$$
$$\beta_{ij}^m = \begin{cases} \beta_j^m, & j \in \mathcal{H}_i \\ *, & j \in \mathcal{B}_i. \end{cases}$$

- I.e., honest clients broadcast their true local stochastic gradient/local model, whereas Byzantine clients send arbitrary vectors denoted by  $*$  called **Byzantine failures**.
- An FL method that defends against Byzantine failures is called **Byzantine-tolerant**.

# Example of Byzantine failure



(a) Clients 1-6 broadcast their true models to the server, and clients 7 and 8 broadcast falsified models.



(b) Byzantine clients shift the aggregated model away from the optimal model.

Figure: Example of Byzantine failure

# Examples of resilient aggregation mechanisms

The two most basic examples of resilient aggregation are median and trimmed mean [YCR18]. In **median**, local stochastic gradients are aggregated

$$\text{median}(g_1^m, \dots, g_K^m).$$

where the median is componentwise. In **trimmed mean**, local stochastic gradients are aggregated

$$\text{trmean}_h(g_1^m, \dots, g_K^m)$$

where the trimmed mean is component wise, and the  $h$ -trimmed mean of scalars  $x_1, \dots, x_K$  is defined as the mean of  $x_1, \dots, x_K$  after its largest  $h$  and smallest  $h$  elements are removed.

## Example of trust-based aggregation mechanism

- In **FLTrust** [CFL20] is a CFL method in which the server has its own local dataset  $D_0$  with which it can compute a local stochastic gradient  $g_0^m \approx \nabla f_0(\beta^m)$ .
- The aggregation rule is

$$\frac{1}{\sum_{i=1}^K t_i^m} \sum_{i=1}^K t_i^m \bar{g}_i^m$$

where the trust score is defined by

$$t_i^m = \text{ReLU} \left( \frac{\langle g_0^m, g_i^m \rangle}{\|g_0^m\| \|g_i^m\|} \right).$$



- 1 Background
- 2 Main Algorithm
- 3 Convergence Analysis
- 4 Numerical Experiments
- 5 Future Research Plans

## 2 Main Algorithm

- Model Update Sequences
- Byzantine-Tolerant Aggregation
- Judgment
- Statement of Algorithm

# Model Updates

So far, model updates have come in one of three “flavors”:

- The non-federated **GD-like update (G)**

$$\beta_i^{m+1} \leftarrow \beta_i^m - \eta^m g_i^m. \quad (\text{G})$$

- The **FedAvg-like update (A)**, in which the aggregation mechanism takes local models as input vectors

$$\beta_i^{m+1} \leftarrow \frac{\eta^m}{\sum_{j \in \mathcal{N}_i} t_{ij}^m} \sum_{j \in \mathcal{N}_i} t_{ij}^m \beta_{ij}^m, \quad (\text{A})$$

which we can rewrite in terms of differences  $d_{ij}^m = \beta_{ij}^m - \beta_i^m$ :

$$\beta_i^{m+1} \leftarrow \beta_i^m + \frac{\eta^m}{\sum_{j \in \mathcal{N}_i} t_{ij}^m} \sum_{j \in \mathcal{N}_i} t_{ij}^m d_{ij}^m. \quad (\text{A})$$

- The **FedSGD-like update (S)**, in which the aggregation mechanism takes local stochastic gradients as input vectors

$$\beta_i^{m+1} \leftarrow \beta_i^m - \frac{\eta^m}{\sum_{j \in \mathcal{N}_i} t_{ij}^m} \sum_{j \in \mathcal{N}_i} t_{ij}^m g_{ij}^m. \quad (\text{S})$$

# Simultaneous and Sequential Model Updates

We can also do multiple model updates simultaneously. For example, **AG update (AG)** and **AS update (AS)**.

$$\beta_i^{m+1} \leftarrow \beta_i^m - \frac{\eta^m}{|\mathcal{N}_i| + \sum_{j \in \mathcal{N}_i} t_{ij}^m} \left( |\mathcal{N}_i| g_i^m - \sum_{j \in \mathcal{N}_i} t_{ij}^m d_{ij}^m \right), \quad (\text{AG})$$

$$\beta_i^{m+1} \leftarrow \beta_i^m - \frac{\eta^m}{2 \sum_{j \in \mathcal{N}_i} t_{ij}^m} \sum_{j \in \mathcal{N}_i} t_{ij}^m (g_{ij}^m - d_{ij}^m). \quad (\text{AS})$$

Finally, we consider multiple model updates sequentially, **“A,G”** and **“G,A”**.

$$\begin{cases} \beta_i^{m+1/2} & \leftarrow \beta_i^m + \frac{\eta^m}{\sum_{j \in \mathcal{N}_i} t_{ij}^m} \sum_{j \in \mathcal{N}_i} t_{ij}^m d_{ij}^m \\ \beta_i^{m+1} & \leftarrow \beta_i^{m+1/2} - \eta^{m+1/2} g_i^{m+1/2}, \end{cases} \quad (\text{“A,G”})$$

$$\begin{cases} \beta_i^{m+1/2} & \leftarrow \beta_i^m - \eta^m g_i^m \\ \beta_i^{m+1} & \leftarrow \beta_i^{m+1/2} + \frac{\eta^{m+1/2}}{\sum_{j \in \mathcal{N}_i} t_{ij}^{m+1/2}} \sum_{j \in \mathcal{N}_i} t_{ij}^{m+1/2} d_{ij}^{m+1/2}. \end{cases} \quad (\text{“G,A”})$$

- 2 Main Algorithm
  - Model Update Sequences
  - Byzantine-Tolerant Aggregation
  - Judgment
  - Statement of Algorithm

# Trust scores are not enough

Consider the naive training loop

- ①  $g_{ij}^m \leftarrow \begin{cases} g_j^m, & j \in \mathcal{H}_i \\ *, & j \in \mathcal{B}_i \end{cases}$  (client estimates local gradient)
- ②  $G_i^m \leftarrow \frac{1}{\sum_{j \in \mathcal{N}_i} t_{ij}^m} \sum_{j \in \mathcal{N}_i} t_{ij}^m g_{ij}^m$  (clients aggregate neighbors' suggestions)
- ③  $\beta_i^{m+1} \leftarrow \beta_i^m - \eta^m G_i^m$ . (clients do FedSGD-like update)

Suppose that for  $j \in \mathcal{N}_i$ , client  $i$  chooses trust score  $t_{ij}^m \in (0, 1)$ . Regardless of client  $i$ 's choice of  $t_{ij}^m$ , a single Byzantine client  $b \in \mathcal{B}_i$  can force  $G_i^m = G_{\mathcal{B}}$  by broadcasting

$$g_{ib}^m = \frac{1}{t_{ib}^m} \left( \left( \sum_{j \in \mathcal{N}_i} t_{ij}^m \right) G_{\mathcal{B}} - \sum_{j \in \mathcal{N}_i \setminus \{b\}} t_{ij}^m g_{ij}^m \right).$$

This is called a **magnitude attack** since  $g_{ib}^m$  has large magnitude.

# Clipping is not enough

The next naive training loop defends against magnitude attacks by employing the clipping operator  $\text{Clip}(x, \tau) = \min\{1, \tau/\|x\|\} \cdot x$ .

- ❶  $g_{ij}^m \leftarrow \begin{cases} g_j^m, & j \in \mathcal{H}_i \\ *, & j \in \mathcal{B}_i \end{cases}$  (clients estimate local gradients)
- ❷  $\tilde{g}_{ij}^m \leftarrow \text{Clip}(g_{ij}^m, \|g_i^m\|)$  (clients clip neighbors' suggestions)
- ❸  $G_i^m \leftarrow \frac{1}{\sum_{j \in \mathcal{N}_i} t_{ij}^m} \sum_{j \in \mathcal{N}_i} t_{ij}^m \tilde{g}_{ij}^m$  (clients aggregate neighbors' clipped suggestions)
- ❹  $\beta_i^{m+1} \leftarrow \beta_i^m - \eta^m G_i^m$  (clients do FedSGD-like update)

From the definition of the clipping operator, we have

$\|\tilde{g}_{ij}^m\| \leq \|g_i^m\|$ , a fact called the **clipping bound**. We show in the write-up that without trust scores, if  $|\mathcal{B}_i| > |\mathcal{H}_i|$ , then coordinated Byzantine clients can still prevent convergence, in spite of the clipping bound.

## 2 Main Algorithm

- Model Update Sequences
- Byzantine-Tolerant Aggregation
- Judgment
- Statement of Algorithm



- In light of the prior discussion, the main algorithm will involve clipping and trust scores.
- Depending on the model update sequence, clients will fetch local stochastic gradients, local models, or both from their neighbors.
- If clients fetch local stochastic gradients, they compute the **angle-based judgment**

$$l_{ij}^{\nabla(m)} \leftarrow \mathbf{1}\{\langle g_i^m, g_{ij}^m \rangle > 0\}.$$

- If clients fetch local models, they compute the **distance-based judgment**

$$l_{ij}^{D(m)} \leftarrow \mathbf{1}\{\|d_{ij}^m\| \leq \|d_{ij}^{m-1}\|\}.$$

- Trust scores are computed

$$t_{ij}^m = \frac{1}{m} \sum_{k=1}^m l_{ij}^k.$$

## 2 Main Algorithm

- Model Update Sequences
- Byzantine-Tolerant Aggregation
- Judgment
- Statement of Algorithm

# High-level overview

During round  $m$ , client  $i \in \mathcal{H}$  does (for  $j \in \mathcal{N}_i$ ):

- 1 **Estimate gradient.** Compute local stochastic gradient  $g_i^m \leftarrow \nabla f_i(\beta_i^m)$ .
- 2 **Fetch messages from your neighbors.** Depending on the model update sequence, fetch message(s)  $g_{ij}^m$  and/or  $\beta_{ij}^m$ .
- 3 **Judge your neighbors.** Depending on whether  $g_{ij}^m$  and/or  $\beta_{ij}^m$  was fetched, compute angle-based judgment  $l_{ij}^{\nabla(m)}$  and/or distance-based judgment  $l_{ij}^{D(m)}$ .
- 4 **Compute trust scores.** Put  $t_{ij}^m \leftarrow \frac{1}{m} \sum_{k=1}^m l_{ij}^k$ .
- 5 **Clip the suggestions of your neighbors.** Clip the suggestion(s)  $g_{ij}^m$  and/or  $d_{ij}^m = \beta_{ij}^m - \beta_i^m$  by  $\|g_i^m\|$ , and obtain **clipped suggestion(s)**  $\tilde{g}_{ij}^m$  and/or  $\tilde{d}_{ij}^m$ .
- 6 **Follow the model update sequence.** Carry out model updates according to the model update sequence and obtain  $\beta_i^{m+1}$ .

To the best of our knowledge our novel contributions are:

- The model update sequences “A,G” and “AS”.
- The model update sequence “S” in a DFL setting.
- Clipping, and trust scores depending on the history of judgments, in a DFL setting.

| Method                   | Topology | Resilient Agg.           | Judgment             | Trust                           | Agg. Seq. | LASSO |
|--------------------------|----------|--------------------------|----------------------|---------------------------------|-----------|-------|
| ClippedGossip            | DFL      | clipping                 | no                   | no                              | “G,A”     | no    |
| FLTrust                  | CFL      | normalize $\nabla$ 's    | cosine               | depends on most recent judgment | “S”       | no    |
| Trusted Decentralized FL | DFL      | no                       | clustering, distance | depends on history of judgments | “G,A”     | no    |
| SLBRN                    | DFL      | median/trimmed mean/Krum | no                   | no                              | “A,S”     | yes   |
| Main Algorithm of Exam   | DFL      | clipping                 | angle, distance      | depends on history of judgments | multiple  | yes   |

Table: Summary of Byzantine tolerant methods

# Complexity analysis

We will for sake of example assume squared loss

$$f_i(\beta) = \frac{1}{2n_i} \|Y_i - X_i\beta\|^2$$

where  $X_i \in \mathbb{R}^{n_i \times p}$ ,  $Y_i \in \mathbb{R}^{n_i}$ , and  $\beta \in \mathbb{R}^p$ .

- 1 The gradient is

$$\nabla f_i(\beta) = \frac{1}{n_i} X_i^\top (X_i\beta - Y_i)$$

with complexity  $O(n_i p)$ . Putting  $n_{\max} := \max_i n_i$ , we have across all clients  $O(n_{\max} p)$ .

- 2 Judgments, clipping, and model updates all involve  $O(p)$  tasks done once per neighbor per round, which is  $O(|\mathcal{N}_i| p)$ . Putting  $\mathcal{N}_{\max} := \max_i |\mathcal{N}_i|$ , we have complexity  $O(\mathcal{N}_{\max} p)$ .

Given the number of rounds  $M$ , the complexity of the main algorithm is  $O(M(n_{\max} + \mathcal{N}_{\max})p)$ .

- In experiments we solve the LASSO problem.
- Each client  $i \in [K]$  possesses  $D_i = (X_i, Y_i)$ , where

$$X_i = \begin{bmatrix} x_{i1} \\ \vdots \\ x_{in_i} \end{bmatrix} \in \mathbb{R}^{n_i \times p}, \quad Y_i = \begin{bmatrix} y_{i1} \\ \vdots \\ y_{in_i} \end{bmatrix} \in \mathbb{R}^{n_i}.$$

- Local objective functions are **LASSO objective functions**

$$f_i(\beta) = F_i(\beta) + \lambda \|\beta\|_1 = \frac{1}{2n_i} \|Y_i - X_i\beta\|^2 + \lambda \|\beta\|_1.$$

- The algorithm is modified in two ways:
  - 1 Replace  $\nabla f_i$  with the gradient of the **differentiable part**  $\nabla F_i$ .
  - 2 Append to each round  $\beta_i^{m+1} \leftarrow S_{\eta^m \lambda}(\beta_i^{m+1})$  where

$$[S_\lambda(x)]_\ell = \begin{cases} x_\ell - \lambda, & x_\ell > \lambda \\ 0, & |x_\ell| \leq \lambda \\ x_\ell + \lambda, & x_\ell < -\lambda. \end{cases}$$

- 1 Background
- 2 Main Algorithm
- 3 Convergence Analysis
- 4 Numerical Experiments
- 5 Future Research Plans

## Definition 1 (Objective function)

The **global objective function** is

$$f(\beta) = F(\beta) + \lambda \|\beta\|_1 = \frac{1}{K} \sum_{i=1}^K F_i(\beta) + \lambda \|\beta\|_1,$$

and the  $F_i$ 's (and hence  $F$ ) are differentiable.

## Definition 2 (Optimal model)

Denote by  $\beta^*$  the minimizer of  $f(\beta)$ .



### Definition 3 (Trust score)

*Trust scores are defined by*

$$t_{ij}^m = \frac{1}{m} \sum_{k=1}^m l_{ij}^m.$$

### Definition 4 (Normalized trust score)

**Normalized trust scores** are defined by  $w_{ij}^m = \frac{t_{ij}^m}{\sum_{j \in \mathcal{N}_i} t_{ij}^m}$  and sum to 1.

### Definition 5 (Aggregated suggestion)

*Under model update sequence “S”, the **aggregated suggestion** is*

$$G_i^m = \sum_{j \in \mathcal{N}_i} w_{ij}^m \tilde{g}_{ij}^m.$$

# Aggregation preserves bounded variance via clipping

## Assumption 1 (Bounded variance)

*The local stochastic gradient satisfies*

$$\mathbb{E} \|g_i^m\|^2 \leq \|\nabla F(\beta_i^m)\|^2 + \sigma^2.$$

## Lemma 1 (Aggregation preserves bounded variance via clipping)

*Under Assumption 1, the aggregated suggestion satisfies*

$$\mathbb{E} \|G_i^m\|^2 \leq \mathbb{E} \|g_i^m\|^2 \leq \|\nabla F(\beta_i^m)\|^2 + \sigma^2.$$

Key ingredient: clipping bound  $\|\tilde{g}_{ij}^m\| \leq \|g_i^m\|$ .

## Assumption 2 (Behavior of neighbors)

- 1 For fixed  $j \in \mathcal{H}_i$ , the judgments  $(I_{ij}^m)_{m \in \mathbb{N}}$  are i.i.d. from  $\text{Bern}(0.5 + \delta)$  for some  $\delta > 0$ .
- 2 For fixed  $j \in \mathcal{B}_i$ , the judgments  $(I_{ij}^m)_{m \in \mathbb{N}}$  are independent and satisfy  $\mathbb{P}(I_{ij}^m = 1) \leq 0.5$ .

## Lemma 2 (Asymptotic trust bounds)

*Under Assumption 2,*

- 1 For  $j \in \mathcal{H}_i$ , we have  $t_{ij}^m \xrightarrow{\text{a.s.}} 0.5 + \delta$ .
- 2 For  $j \in \mathcal{B}_i$ , we have  $\limsup_{m \rightarrow \infty} t_{ij}^m \leq 0.5$  almost surely.

Key ingredients: SLLN for honest case, Hoeffding's inequality and Borel-Cantelli lemma for Byzantine case.

## Assumption 3 (Honest alignment)

*There exists  $C > 0$  such that for  $j \in \mathcal{H}_i$ ,*

$$\mathbb{E}\langle \tilde{g}_{ij}^m, \nabla F(\beta_i^m) \rangle \geq C \|\nabla F(\beta_i^m)\|.$$

## Assumption 4 ( $L$ -smoothness)

*There exists  $L > 0$  such that for all  $\beta, \beta' \in \mathbb{R}^p$ ,*

$$\|\nabla F(\beta) - \nabla F(\beta')\| \leq L \|\beta - \beta'\|.$$

## Assumption 5 (Bounded gradient)

*For  $\beta$  along the optimization path, the norm of the gradient is bounded by*

$$\|\nabla F(\beta)\| \leq D = LR = L\|\beta^0 - \beta^*\|.$$

## Expected descent direction (cont.)

### Assumption 6 (Bounded Byzantine ratio)

*The Byzantine ratio is bounded by*

$$\frac{|\mathcal{B}_i|}{|\mathcal{H}_i|} \leq \frac{(0.5 + \delta)C}{0.5(D + \sigma)}.$$

### Lemma 3 (Expected descent direction)

*Under Assumptions 1, 2, 3, 5, and 6,*

$$\langle \mathbb{E} G_i^m, \nabla F(\beta_i^m) \rangle \geq 0.$$

Sketch: Rewrite the inner product as the sum of an honest term and a Byzantine term. The Byzantine term is bounded using the clipping bound and Assumption 1. The honest term is bounded by Assumption 3. Using Lemma 2 we can rewrite trust scores in terms of  $|\mathcal{H}_i|$  and  $|\mathcal{B}_i|$ . The final result follows by substituting in Assumptions 5 and 6.

## Assumption 7 ( $\mu$ -strong convexity)

For all  $\beta, \beta' \in \mathbb{R}^p$ ,

$$\langle \beta - \beta', \nabla F(\beta) - \nabla F(\beta') \rangle \geq \mu \|\beta - \beta'\|^2.$$

## Theorem 1 (Convergence)

Under Assumptions 1, 2, 3, 4, 5, 6, and 7, and with constant step size  $0 < \eta < \min \left\{ \frac{\mu}{4L^2}, \frac{2}{\mu} \right\}$ ,

$$\mathbb{E} \|\beta_i^m - \beta^*\|^2 \leq \left(1 - \frac{\eta\mu}{2}\right)^m \|\beta_i^0 - \beta^*\|^2 + \frac{10}{\mu^2} (2L^2 \|\beta^0 - \beta^*\|^2 + \sigma^2).$$

# Proof sketch for main convergence result

First, we bound  $\|\beta_i^{m+1} - \beta^*\|^2$  above by

$$\leq \|\beta_i^m - \beta^*\|^2 + \eta^2 \|G_i^m - \nabla F(\beta^*)\|^2 - 2\eta \langle \beta_i^m - \beta^*, G_i^m - \nabla F(\beta^*) \rangle.$$

The purple term can be bounded with  $L$ -smoothness and the green term with  $\mu$ -strong convexity, yielding

$$\|\beta_i^{m+1} - \beta^*\|^2 \leq \left(1 - \frac{\eta\mu}{2}\right) \|\beta_i^m - \beta^*\|^2 + \left(\frac{\eta}{\mu} + 2\eta^2\right) \|\Delta_i^m\|^2.$$

Unrolling the geometric sequence yields

$$\mathbb{E} \|\beta_i^m - \beta^*\|^2 \leq \left(1 - \frac{\eta\mu}{2}\right)^m R^2 + \frac{10}{\mu^2} \mathbb{E} \|\Delta_i^m\|^2.$$

Finally, we expand

$\mathbb{E} \|\Delta_i^m\|^2 = \mathbb{E} \|G_i^m\|^2 - 2\langle \mathbb{E} G_i^m, \nabla F(\beta_i^m) \rangle + \|\nabla F(\beta_i^m)\|^2$  and bound the purple term with Lemma 1 and the green term with Lemma 3.

# Roadmap

- 1 Background
- 2 Main Algorithm
- 3 Convergence Analysis
- 4 Numerical Experiments**
- 5 Future Research Plans



# Data generation

- Across all four experiments, each client has the same sample size  $n_1 = \dots = n_K = 50$  and dimension  $p = 100$ .
- For  $i \in [K]$ , the outcome  $Y_i \in \mathbb{R}^{50}$  is generated from the linear regression model

$$Y_i = X_i \beta^* + \epsilon_i$$

for  $X_i \in \mathbb{R}^{100 \times 50}$ ,  $\beta^* \in \mathbb{R}^{50}$ , and  $\epsilon_i \in \mathbb{R}^{100}$ .

- The true model  $\beta^*$  is a vector of 95 zeros and 5 ones, the positions of the ones drawn uniformly without replacement from  $[100]$ .
- Each row of each  $X_i$ ,  $i \in [K]$  is drawn i.i.d. from  $N_{100}(0_{100}, I_{100})$ .
- Each entry of each  $\epsilon_i$ ,  $i \in [K]$  is drawn i.i.d. from  $N(0, \|\beta^*\|^2)$  (i.e.,  $\text{SNR} = 1$ ).

# Objective function

- Since  $Y_i$  is generated from a linear regression model and  $\beta^*$  is sparse, local objective functions are LASSO objective functions

$$f_i(\beta) = \frac{1}{100} \|Y_i - X_i \beta\|^2 + \lambda \|\beta\|_1.$$

- The global objective function is

$$f(\beta) = \frac{1}{K} \sum_{i=1}^K \frac{1}{100} \|Y_i - X_i \beta\|^2 + \lambda \|\beta\|_1 = \frac{1}{100K} \|Y - X \beta\|^2 + \lambda \|\beta\|_1,$$

where the **global dataset** is

$$X = \begin{bmatrix} X_1 \\ \vdots \\ X_K \end{bmatrix} \in \mathbb{R}^{50K \times 100}, \quad Y = \begin{bmatrix} Y_1 \\ \vdots \\ Y_K \end{bmatrix} \in \mathbb{R}^{50K}.$$

- The global objective function is itself a LASSO objective function, hence optimizable with proximal gradient descent.

# Non-federated benchmarks

- The data are generated such that each local dataset  $(X_i, Y_i)$  does not contain enough samples for a non-federated method (“G”) to perform well.
- By contrast, the global dataset  $(X, Y)$ , which is not accessible in the FL setting, contains enough samples for “G” to perform well.
- We include among the competitors two “non-federated benchmarks”.
- “G” (local) uses (non-federated) proximal gradient descent to optimize

$$f_i(\beta) = \frac{1}{100} \|Y_i - X_i \beta\|^2 + \lambda \|\beta\|_1.$$

- “G” (global) uses (non-federated) proximal gradient descent to optimize

$$f(\beta) = \frac{1}{100K} \|Y - X\beta\|^2 + \lambda \|\beta\|_1.$$

# Byzantine clients

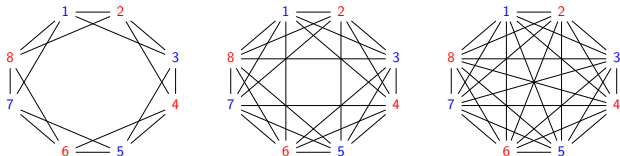
We consider three different kinds of Byzantine failures. Given a **Byzantine flip proportion**  $p_{\text{flip}} \in [0, 1]$  a Byzantine client will flip the sign of  $p_{\text{flip}}$  proportion of the entries of  $X_i$ ,  $Y_i$ , or  $g_i$ . Having done so, Byzantine clients will perform “G” on their poisoned data, and broadcast their poisoned local gradient and local model to their neighbors.

|          | Before flipping                                                      | After flipping                                                                                                                                                              |
|----------|----------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>X</b> | $X_i = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \end{bmatrix}$ | $X_{i,\text{flip}} = \begin{bmatrix} 1 & -2 & -3 & 4 \\ -5 & 6 & -7 & 8 \end{bmatrix}; \quad g_{ij} = \frac{1}{n_i} X_{i,\text{flip}}^\top (X_{i,\text{flip}} \beta - Y_i)$ |
| <b>Y</b> | $Y_i = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$                         | $Y_{i,\text{flip}} = \begin{bmatrix} -1 \\ 2 \end{bmatrix}; \quad g_{ij} = \frac{1}{n_i} X_i^\top (X_i \beta - Y_{i,\text{flip}})$                                          |
| <b>g</b> | $g_i = \begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \end{bmatrix}$               | $g_{ij} = \begin{bmatrix} 1 \\ -2 \\ -3 \\ 4 \end{bmatrix}$                                                                                                                 |

**Table:** Depiction of 3 different poisoning attacks, where  $i \in \mathcal{B}$ ,  $n_i = 2$ ,  $p = 4$ , and  $p_{\text{flip}} = 1/2$ .

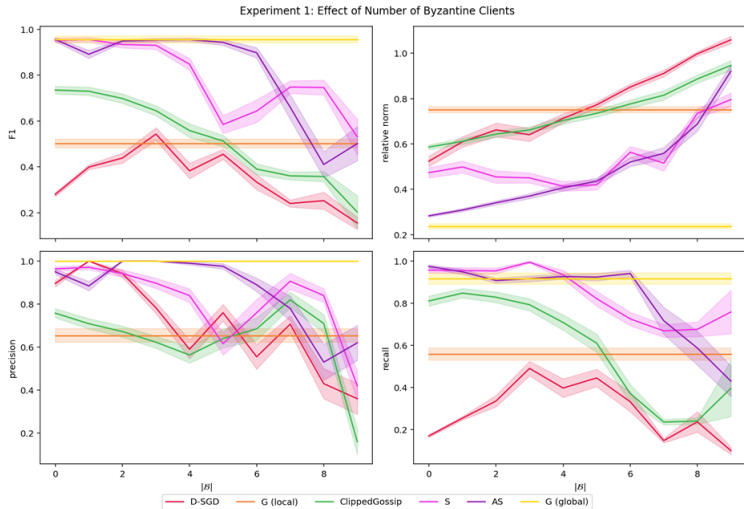
# Communication graph

Three of the experiments take place on the **mesh**, i.e. complete graph. One of the experiments takes place on the **band** with varying **connectivity**.

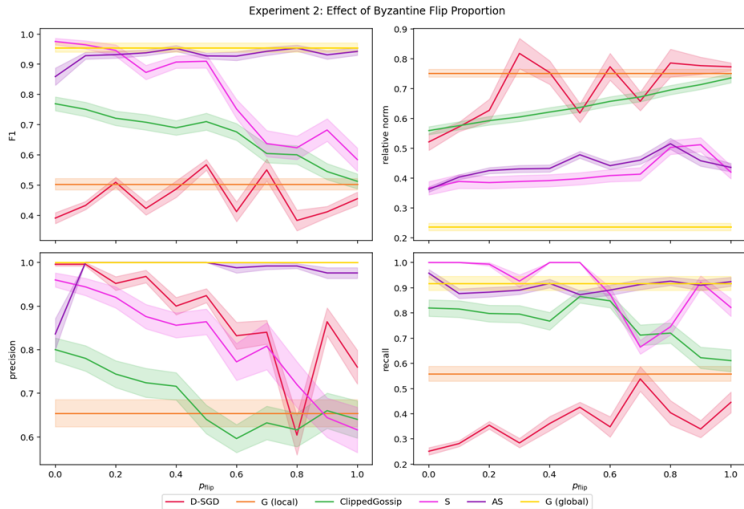


**Figure:** From left to right: the band with  $K = 8$  and  $\tau = 2$ ; the band with  $K = 8$  and  $\tau = 3$ ; the band with  $K = 8$  and  $\tau = 4$ , equivalently the mesh with  $K = 8$ . Odd-numbered clients are honest, and even-numbered clients are Byzantine.

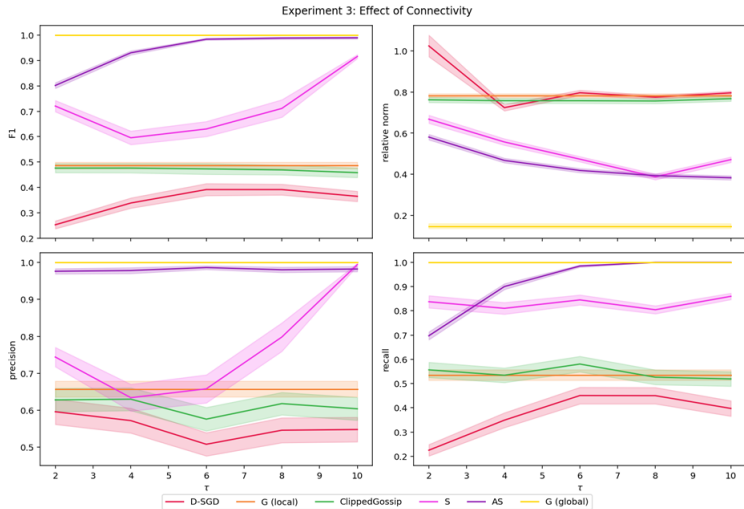
# Experiment 1



# Experiment 2



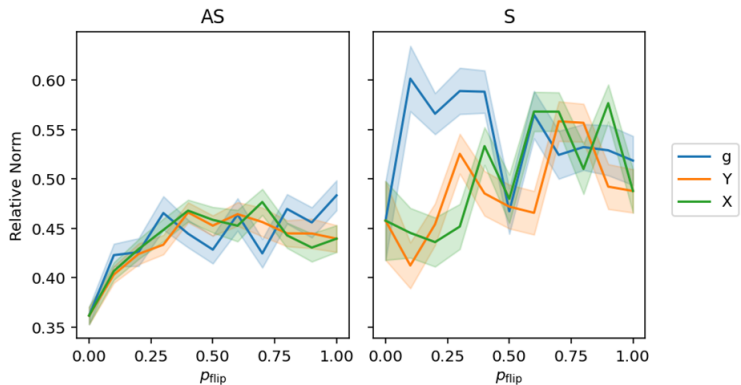
# Experiment 3





# Experiment 4

Experiment 4: Effect of Byzantine Failure Type



# Roadmap

- 1 Background
- 2 Main Algorithm
- 3 Convergence Analysis
- 4 Numerical Experiments
- 5 Future Research Plans

# Overview of graphical models

Let  $X = (X_1, \dots, X_p) \in \mathbb{R}^p$  be a random vector with positive density. Let  $\mathcal{G}_\perp = ([p], \mathcal{E}_\perp)$  be an undirected graph, where each  $X_i$  is identified with the vertex  $i$ . The graph  $\mathcal{G}_\perp$  is a **conditional independence graph** if for any two vertices  $i \neq j \in [p]$ ,

$$\{i, j\} \notin \mathcal{E}_\perp \implies i \perp j \mid [K] \setminus \{i, j\}.$$

If additionally  $X$  is multivariate normal, we say  $\mathcal{G}_\perp$  is a **Gaussian graphical model**. Without loss of generality, we assume  $X \sim N_p(0_p, \Omega^{-1})$ , where  $\Omega \in \mathbb{R}^{p \times p}$  is the **precision matrix**. We are interested in learning the edge set  $\mathcal{E}_\perp$  of the graphical model, since it encodes conditional independence relationships between the  $X_i$ 's.

# Precision matrix estimation

The next proposition says that learning this edge set is tantamount to learning the support of  $\Omega$ .

## Proposition 1 (adjacency and precision share support)

*If  $\mathcal{G}_\perp = ([p], \mathcal{E}_\perp)$  is a Gaussian graphical model with respect to  $X \sim N_p(0_p, \Omega^{-1})$ , then  $\{i, j\} \in \mathcal{E}_\perp$  if and only if  $\Omega_{ij} \neq 0$ .*

The following proposition shows that the  $\Omega$  can be recovered via regression.

## Proposition 2 (support of precision as regression coefficients)

*Define the matrix  $\beta \in \mathbb{R}^{p \times p}$  by  $\beta_{ij} = -\Omega_{ij}/\Omega_{jj}$  and  $\beta_{ii} = 0$ . If  $X = [X_1 \dots X_p] \sim N_p(0, \Omega^{-1})$ , then  $X_j$  given  $X_{-j}$  is Gaussian with*

$$\mathbb{E}[X_j \mid X_{-j}] = - \sum_{i \neq j} \beta_{ij} X_i$$

$$\text{Var}(X_j \mid X_{-j}) = \Omega_{jj}^{-1}.$$

# Neighborhood regression

Proposition 2 along with a sparsity assumption is the setup for  $p$  LASSO problems. **Neighborhood regression** [MB06] solves these  $p$  LASSO problems as described in Algorithm 1. Let  $X \in \mathbb{R}^{n \times p}$ , where the rows of  $X$  are i.i.d. draws from  $N_p(0_p, \Omega^{-1})$ .

---

**Algorithm** Neighborhood regression

---

**for**  $\ell \in [p]$  **do**

    Solve the LASSO problem

$$\hat{\beta}_\ell = \arg \min_{\beta \in \mathbb{R}^{p-1}} \left\{ \frac{1}{2n} \|X_{\cdot\ell} - (X_{\cdot-\ell})^\top \beta\|^2 + \lambda \|\beta\|_1 \right\}$$

    Insert a 0 into  $\hat{\beta}_\ell$  at the  $\ell$ -th index. Now  $\hat{\beta}^\ell \in \mathbb{R}^p$  and the  $\ell$ -th component is 0.

Put  $\hat{\beta} \leftarrow [\hat{\beta}_1 \dots \hat{\beta}_p] \in \mathbb{R}^{p \times p}$

**return**  $\hat{A} \in \{0, 1\}^{p \times p}$  defined by

$$[\hat{A}]_{kl} \leftarrow \mathbf{1}\{\hat{\beta}_{kl} \neq 0 \text{ AND/OR } \hat{\beta}_{lk} \neq 0\}.$$

# Byzantine decentralized neighborhood regression

- In the Byzantine DFL setting, there are clients  $i \in [K]$  each holding  $X^{(i)}$ , and  $X^{(1)}, \dots, X^{(K)} \stackrel{\text{iid}}{\sim} N_p(0_p, \Omega^{-1})$ . The  $K$  clients want to collaborate to learn  $\Omega$  without sharing their private datasets  $X^{(i)}$ . Moreover, some Byzantine clients will introduce Byzantine failures rather than collaborating honestly.
- To handle the precision matrix estimation problem in the Byzantine DFL setting, we can run  $p$  synchronous instances of the main algorithm, each solving one of the  $p$  LASSO problems of neighborhood regression.
- These  $p$  instances are independent except for sharing trust scores. I.e., let  $I_{ij}^{\ell m}$  denote client  $i$ 's judgment of neighbor  $j$  during round  $m$  of the  $\ell$ -th LASSO problem. Then, across all  $p$  LASSO problems, the trust score client  $i$  assigns to neighbor  $j$  during round  $m$  is

$$t_{ij}^m \leftarrow \frac{1}{mp} \sum_{k=1}^m \sum_{\ell=1}^p I_{ij}^{\ell m}.$$

- In **personalized federated learning (PFL)**, the global objective function is

$$f(\beta_1, \dots, \beta_K) = \frac{1}{K} \sum_{i=1}^K f_i(\beta_i).$$

- Each client wants to learn its own “personal”  $\beta_i$  minimizing its local objective function.
- Clients might be interested in minimizing such an objective function under data heterogeneity, i.e., for  $i \in [K]$ ,  $D_i \sim \mathcal{D}_i$ , where the  $\mathcal{D}_i$ ’s are distinct distributions.
- If the  $\mathcal{D}_i$ ’s are completely arbitrary, then the best solution is for each client to solve its own local problem without any collaboration.
- Hence, we assume that clients’ data are heterogeneous in some structured way.

# PFL strategies

PFL strategies fall into two categories.

- **Global model personalization** involves training a global model with CFL, and then personalizing the global model for each client through a local adaptation step involving additional training on local datasets.
- **Learning personalized models** involves training personalized models to begin with.
- Among “learning personalized models” PFL methods are **similarity-based methods**, in which related clients learn similar models.
- Trust-based DFL methods are similar to similarity-based PFL methods, except dissimilarity between clients is induced by Byzantine failures rather than heterogeneous data.
- The main algorithm works when the judgments  $I_{ij}^{\nabla(m)}$  and  $I_{ij}^{D(m)}$  detect Byzantine clients more than half the time. If these judgments can analogously detect heterogeneous clients, then the main algorithm might be suitable for PFL.



# Pseudocounts

- The current algorithm can address such prior knowledge using “pseudocounts”. Currently, trust scores are parameterized

$$t_{ij}^m = \frac{\sum_{k=1}^m I_{ij}^k}{m}.$$

- We could instead define

$$a_{ij}^m = a_{ij}^0 + \sum_{k=1}^m I_{ij}^k, \quad b_{ij}^m = b_{ij}^0 + \sum_{k=1}^m (1 - I_{ij}^k)$$
$$t_{ij}^m = \frac{a_{ij}^m}{a_{ij}^m + b_{ij}^m} = \frac{a_{ij}^0 + \sum_{k=1}^m I_{ij}^k}{a_{ij}^0 + b_{ij}^0 + m},$$

where  $a_{ij}^0$  and  $b_{ij}^0$  are called **pseudocounts**.

- If client  $i$  believes it and neighbor  $j$  are similar, client  $i$  can initialize a high value of  $t_{ij}^0 = \frac{a_{ij}^0}{a_{ij}^0 + b_{ij}^0}$ .
- If client  $i$  is very certain in its prior belief, it can initialize high values of  $a_{ij}^0$  and  $b_{ij}^0$ , lessening the effect that subsequent judgments have on the trust score every round.

- June: Improvements to convergence proof: extend to “AG”, “A,G”, “G,A”, “AS”; non-constant step size
- October: Investigate GLMs
- November: Investigate graphical models
- February: Investigate PFL



Xiaoyu Cao, Minghong Fang, Jia Liu, and Neil Zhenqiang Gong.

“FLTrust: Byzantine-robust Federated Learning via Trust Bootstrapping.”

*CoRR*, [abs/2012.13995](#), 2020.



Anastasia Koloskova, Nicolas Loizou, Sadra Boreiri, Martin Jaggi, and Sebastian U. Stich.

“A Unified Theory of Decentralized SGD with Changing Topology and Local Updates.”, 2021.



Xiangru Lian, Ce Zhang, Huan Zhang, Cho-Jui Hsieh, Wei Zhang, and Ji Liu.

“Can Decentralized Algorithms Outperform Centralized Algorithms? A Case Study for Decentralized Parallel Stochastic Gradient Descent.”, 2017.



Nicolai Meinshausen and Peter Bühlmann.

“High-dimensional graphs and variable selection with the Lasso.”

*The Annals of Statistics*, **34**(3), 2006.



H. Brendan McMahan, Eider Moore, Daniel Ramage, and Blaise Agüera y Arcas.

“Federated Learning of Deep Networks using Model Averaging.”

*CoRR*, [\*\*abs/1602.05629\*\*](#), 2016.



Dong Yin, Yudong Chen, Kannan Ramchandran, and Peter L. Bartlett.

“Byzantine-Robust Distributed Learning: Towards Optimal Statistical Rates.”

*CoRR*, [\*\*abs/1803.01498\*\*](#), 2018.